Unit III

3. Intensity Transformation and Spatial Filtering

3.1 Background : Intensity Transformation Functions
3.2 Using imadjust( ), Using log( )
3.3 Histogram Processing and function plotting, Spatial filtering
3.4 Linear spatial filtering, Non-Linear spatial filtering

3.1 Background : Intensity Transformation Functions

Intensity Transformation Functions are basic operations in Digital Image Processing used to modify the pixel intensity values of an image. These functions apply a mathematical transformation to each pixel to improve image appearance or prepare it for further processing. They operate directly on the intensity values without changing the pixel positions.

Key Points About Intensity Transformation Functions:

1. **Definition:**
   Intensity transformation functions map input pixel values (**r**) to new output values (**s**) using a transformation function **s = T(r)**. Each pixel is modified independently.

2. **Purpose:**
   They are used to enhance contrast, brighten or darken images, correct lighting issues, highlight certain details, and prepare images for segmentation or analysis.

3. **Common Types of Transformations:**
   Linear Transformations: Brightness adjustment, contrast stretching.
   Logarithmic Transformations: Enhance dark regions and compress bright regions.
   Power-Law (Gamma) Transformations: Control overall image brightness using gamma correction.
   Piecewise Linear Transformations: Includes thresholding and gray-level slicing.

4. **Point Processing:**
   Intensity transformations are point-processing operations, meaning each output pixel depends only on the corresponding input pixel, making them simple and fast.

5. **Applications:**
   Used in medical imaging, satellite images, photography, and preprocessing steps before edge detection, segmentation, and pattern recognition.

Intensity transformation functions play a fundamental role in enhancing image quality and improving visibility of important details in digital images.

3.2 Using imadjust( ), Using log( ):

1. Using `imadjust()`:

`imadjust()` is a MATLAB function used to enhance the contrast of an image by adjusting the intensity values. It performs **intensity mapping** by stretching or compressing the range of pixel values.

Key Points:

1. **Definition:**
   `imadjust()` maps the intensity values of an image to a new range, improving brightness and contrast. The transformation is of the form:
   s = T(r), where `r` is input intensity and `s` is output intensity.

2. **Syntax:**
   `J = imadjust(I, [low_in high_in], [low_out high_out])`

3. **Functionality:**
   Pixels below `low_in` are mapped to `low_out`.
   Pixels above `high_in` are mapped to `high_out`.
   Pixels between them are linearly stretched.

4. **Uses:**
   Brightening dark images
   Increasing contrast of low-contrast images
   Normalizing uneven lighting
   Adjusting specific intensity ranges

5. **Example:**
   `J = imadjust(I, [0.2 0.8], [0 1]);`
   This stretches the contrast by expanding the middle intensities into the full 0–1 range.

`imadjust()` is widely used for preprocessing before segmentation, enhancement, and edge detection.

2. Using `log()` (Logarithmic Transformation):

Logarithmic transformation enhances the details in **dark regions** of an image while compressing the brightness of **high-intensity pixels**. It is useful for images with **large dynamic ranges** (very bright and very dark areas in the same image).

Key Points:

1. **Definition:**
   A log transformation applies the formula:
   $$\mathbf{s = c \cdot \log(1 + r)}$$
   where `r` is the pixel intensity and `c` is a scaling constant.

2. **Purpose:**
   Expands low-intensity values (makes dark areas brighter)
   Compresses high-intensity values (reduces brightness of very bright areas)
   Enhances details in shadows

3. **MATLAB Usage:**
   ```
   I = im2double(I);
   J = c * log(1 + I);
   ```

   `c` is chosen to normalize the output.

4. **Applications:**
   Medical imaging (X-rays, CT scans)
   Satellite images (night-time, low illumination)
   Images with high contrast variations
   Detecting details hidden in dark regions

5. **Characteristics:**
   Non-linear transformation
   Useful for compressing large dynamic ranges
   Often applied with normalization for best visual results

Log transformation is a powerful tool for enhancing low-intensity regions and is a standard technique in image enhancement.

3.3 Histogram Processing and function plotting, Spatial filtering:

1. Histogram Processing:

Histogram processing is a fundamental technique in digital image processing used to analyze and enhance the contrast of an image based on the distribution of its pixel intensities.

Definition:

A histogram is a graphical representation of the frequency of intensity values in an image. It shows how many pixels correspond to each gray level.

Key Concepts:

1. **Histogram Equalization:**
   Enhances contrast by spreading out frequently occurring intensity values. It produces an image with uniform histogram distribution.

2. **Histogram Matching (Specification):**
   Adjusts the image so that its histogram matches a desired target histogram.

3. **Histogram Stretching (Normalization):**
   Expands the range of intensity values to cover the full dynamic range (e.g., 0–255), improving contrast.

4. **Cumulative Distribution Function (CDF):**
   Used in calculating equalized intensity values.

Uses:

Improving visibility of details in low-contrast images
Medical imaging, satellite imaging, pattern recognition
Preprocessing for further image analysis

Conclusion:

Histogram processing is widely used to enhance image contrast and reveal important features that may not be visible in the original image.

2. Function Plotting:

Function plotting in MATLAB is used to visualize mathematical expressions or relationships between variables.

Key Points:

1. **Definition:**
   It refers to graphically representing a function (y=f(x)) using MATLAB commands.

2. **Basic Plot Command:**

```
x = 0:0.1:10;
y = sin(x);
plot(x,y)
```

   This generates a 2D plot of the sine function.

3. **Labels and Titles:**
   Commands like `xlabel`, `ylabel`, `title`, and `grid` improve readability.

4. **Multiple Plots:**
   MATLAB allows plotting several functions on the same graph using `hold on` or by specifying multiple parameters in `plot()`.

5. **Applications:**
   Used in mathematics, engineering analysis, signal processing, and algorithm visualization.

Conclusion:
Function plotting helps understand trends, compare functions, and visually interpret mathematical results.

3. Spatial Filtering:
Spatial filtering is an image processing operation that modifies pixel intensity values based on the values of neighboring pixels.

Definition:
A filter (also called a mask or kernel) is applied to an image by sliding it across the image and computing a new value for each pixel.

Types of Spatial Filters:

1. **Linear Filters (Convolution-based):**
   Smoothing/Blur Filters: Reduce noise and details using averaging or Gaussian filters.
   Sharpening Filters: Highlight edges using Laplacian or high-pass filters.

2. **Nonlinear Filters:**
   Median Filter: Removes salt-and-pepper noise by taking the median of neighbourhood pixels.
   Max/Min Filters: Used in morphological operations.

Operations Used:

Convolution:
Applying the filter weights to corresponding image pixels.

Correlation:
Similar to convolution but without kernel flipping.

Applications:
Noise reduction
Edge detection
Image smoothing and sharpening
Feature enhancement

Conclusion:
Spatial filtering improves image quality and extracts important features, making it essential in many image analysis tasks.

3.4 Linear spatial filtering, Non-Linear spatial filtering:

1. Linear Spatial Filtering:
Linear spatial filtering is an image processing technique in which the output pixel value is a **linear combination of the neighboring pixel intensities**. It uses a filter (kernel or mask) that is applied over the image through **convolution** or **correlation**.

Key Points:

1. **Definition:**

   A linear filter replaces each pixel with a weighted sum of its neighborhood. The weights are defined by a filter mask.

2. **Convolution Operation:**

   The mask moves over the image, multiplies its values with the underlying pixels, and adds the results to produce a new pixel value.

   Example:

$$g(x,y) = \sum i \sum j \; w(i,j) f(x-i, y-j)$$

3. **Types of Linear Filters:**

   Smoothing (Low-pass) Filters: Reduce noise and blur the image.

   Examples: Averaging filter, Gaussian filter.

   Sharpening (High-pass) Filters: Enhance edges and fine details.

   Examples: Laplacian filter, Sobel and Prewitt operators.

4. **Applications:**

   Noise reduction
   Edge detection
   Image enhancement
   Smoothing or blurring

Conclusion:

Linear spatial filtering is widely used because it is mathematically simple, easy to implement, and effective for tasks like smoothing and sharpening images.

2. Non-Linear Spatial Filtering:

Non-linear spatial filtering processes an image using nonlinear operations, meaning the output pixel is **not** a linear combination of neighbourhood pixels. These filters are especially useful for noise removal while preserving edges.

Key Points:

1. **Definition:**

   A non-linear filter replaces each pixel with a value derived from its neighborhood using a **nonlinear** mathematical function (median, max, min, etc.).

2. **Common Non-linear Filters:**

   Median Filter: Replaces pixel with median of neighborhood. Very effective for salt and-pepper noise.

   Max Filter: Selects highest intensity in the window (used in dilation).

   Min Filter: Selects lowest intensity in the window (used in erosion).

   Adaptive Filters: Adjust based on local image statistics.

3. **Advantages:**

   Preserves edges better than linear smoothing filters
   Removes impulsive noise effectively

4. **Applications:**

   Removing salt-and-pepper noise
   Edge-preserving smoothing
   Morphological image processing
   Preprocessing for segmentation

Conclusion:

Non-linear spatial filtering is powerful for noise removal and edge preservation, making it essential in applications where linear filters fail to maintain important image details.